

D1.5 - Design Explanation:

D1.5a - Class Responsibilities:

- **Booking.java:**
 - Object that represents a booking record
 - **String: id, String: spaceld, int: duration, int: version, boolean: accessibilityRequirement.**
 - Constructor runs DomainRules methods to validate that id, spaceld, and duration are valid so that an invalid Booking cannot be created
 - Contains getter methods for all fields
- **Space.java:**
 - Object that represents individual parking spaces
 - **String: id, boolean: open, boolean: occupied, boolean: accessible**
 - Constructor runs DomainRules methods to validate that id is valid so that an invalid Space cannot be created
 - **boolean isAvailable()** determines if space is available. Returns true if the open field is true and the occupied field is false. Returns false otherwise.
- **Proposal.java:**
 - Object that represents pending space assignment requests.
 - **String: id, String: bookingId, String: spaceld, int: bookingVersion, int: policyVersion, ProposalStatus status**
 - Constructor runs DomainRules methods to validate that id, bookingId, and spaceld so that an invalid Proposal cannot be created
 - Constructor checks if booking version < 0 and policy version < 1 and throws IllegalArgumentException if condition(s) are not met
 - Initially, status is in PENDING state
 - Contains getter methods for all fields.
- **StudentApplication.java:**
 - Main entry point for ParkOps. Holds a booking, and a set of parking spaces.
 - The main purpose (as of D1) is to create proposals to move bookings into different spaces.
 - Constructor builds booking and parking spaces from data from Fixture.
 - **Map<String, Object> bookingSnapshot()** returns a booking object as a map where field names are the keys and values are the booking field data
 - **Map<String, Object> spaceSnapshot(String spaceld)** does the same thing as bookingSnapshot but for spaces. Throws IllegalArgumentException if given space does not exist
 - **int proposalCount()** returns the current number of proposals
 - **ProposalView propose(String targetId)** creates and records a proposal after checking if the target space is valid, exists, is available, and not the bookings current space.
 - **String run(ModelProvider provider, int limit)** is not yet implemented, will be implemented in D2

- **ProposalStatus.java:**
 - Enumeration class that contains four proposal status types
 - **PENDING, APPROVED, REJECTED, EXECUTED**
 - Used in Proposal class
- **DomainRules.java:**
 - A set of methods that are called to validate hours and ID
 - Called by Booking, Space, Proposal, and StudentApplication classes
 - **void RequireDuration(int hours)** ensures that a given integer representing a number of hours is between 1 and 24. Throws an IllegalArgumentException otherwise.
 - **void requireIdentifier(String id)** ensures that a given string which represents an identifier follows the specific format [A-Z][A-Z0-9_-]{0,15}. Throws an IllegalArgumentException if the string is null or not in format.

D1.5b - Alternative Considered:

In StudentApplication.java, the starter code had the booking field as a LinkedHashMap object that is copied from Fixture.booking(). I thought about keeping it the way it was but instead I decided to go with an alternate approach because the map is able to store an invalid duration or id value and on every read would need an unchecked cast. My alternative decision was to change the booking field to a single Booking object and then implement the bookingSnapshot() method which returns a Map.of(booking) so that the caller receives an unmodifiable version.

D1.5c - Operation Contracts:

1. **new Booking(String: id, String: spaceId, int: duration, int: version, boolean: accessibilityRequirement)**
 - **Preconditions:** None
 - **Success:** id, spaceId, and duration are valid according to DomainRules
 - Booking object is successfully constructed
 - **Failure:** One or more of id, spaceId, or duration is invalid according to DomainRules, IllegalArgumentException is thrown
 - Booking object is not constructed.
2. **StudentApplication.propose(String: targetId)**
 - **Preconditions:** none
 - **Success:** targetId is valid according to DomainRules, there exists a space with the same id as targetId that is not the same as booking.getSpaceId() and target space is available.
 - New proposal is constructed based on the valid targetId, the new proposal is added to proposals, and returned as a ProposalView object
 - **Failure:** Throws an IllegalArgumentException if targetId is invalid according to DomainRules, targetId references an unknown space, is the same as booking.getSpaceId(), or target space is not available.
 - If any above are true, proposal is not constructed and proposals is unchanged

D1.5d - Current Limitation:

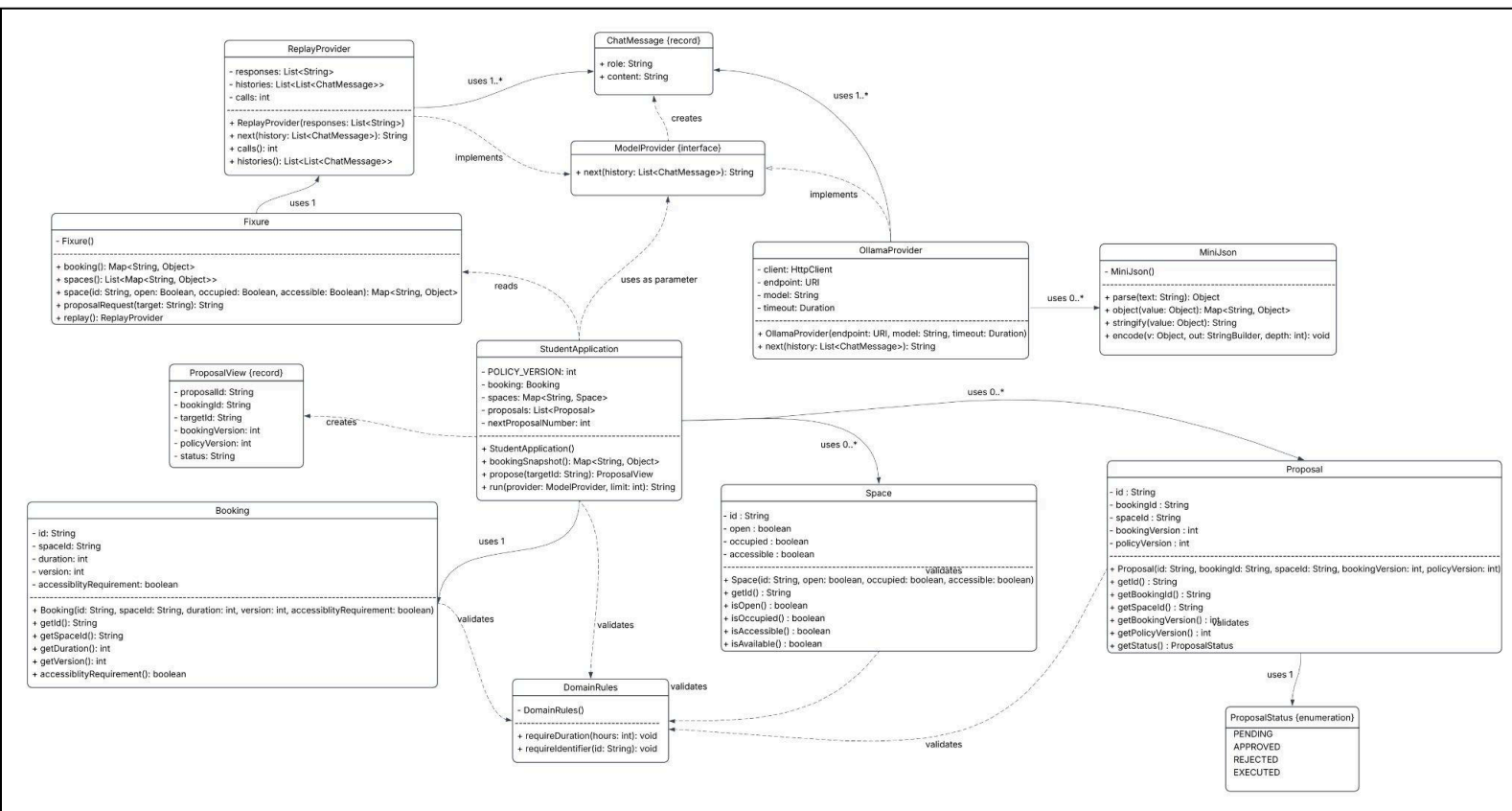
Currently, in Proposal.java, the ProposalStatus status field is initially set by the constructor to have state PENDING. The ProposalStatus enumeration has three other status types APPROVED, REJECTED, and EXECUTED which are currently unreachable.

D1.5e - Reused and Implemented Infrastructure:

- **Reused (unchanged):**
 - ChatMessage.java, Fixture.java, MiniJson.java, ModelProvider.java, OllamaProvider.java, ProposalView.java, ReplayProvider.java
- **Implemented (changed/added):**
 - Booking.java, Space.java, Proposal.java, ProposalStatus.java.
 - StudentApplication(), StudentApplication.bookingSnapshot(), StudentApplication.spaceSnapshot(), StudentApplication.proposalCount(), StudentApplication.propose()
 - DomainRules.requireDuration(), DomainRules.requireIdentifier()
 - Junit test cases in test/junit/StudentTests.java
 - Executable path exercise in Main.java

D1.5f - Diagrams:

- **UML Diagram:**



- Sequence diagram:

